

Peer-Review 1: UML

Isabella Argentiero, Lorenzo Castellana, Samuele Celedi, Matteo Montanelli

Gruppo AM10

Valutazione del diagramma UML delle classi del gruppo AM19

Lati positivi

- L'outline del modello appare generalmente corretto, sebbene ci siano alcuni piccoli errori concettuali. Si nota un utilizzo della sintassi corretta di UML, particolarmente le relazioni e la cardinalità.
- Si nota una buona gestione degli angoli, tuttavia è necessario implementare correttamente il calcolo dei punti.

Lati negativi

Abbiamo deciso di separare i problemi generali da quelli relativi a classi, metodi o implementazioni.

Commento generale:

Nonostante la chiarezza delle relazioni tra classi/entità, il diagramma risulta poco chiaro per quanto riguarda le funzioni interne. Vi è una grande carenza di metodi basilari come setter e getter.

Ci è stato inoltre difficile capire l'UML in quanto per la maggior parte dei metodi e delle variabili non è segnato il tipo ma solo il nome della variabile/funzione.

Notiamo inoltre che non sembra essere stato rispettato correttamente il design pattern Model-View-Control, guardando in particolare come comunicano Player e PlayerCardsArea (Player nel controller andrebbe ad inizializzare ed usare una classe del model).

Consigliamo inoltre di evitare nomi eccessivamente lunghi per i metodi, in quanto possono generare confusione e difficoltà di lettura. Sarebbe meglio dare nomi più significativi.

Elenchiamo qui di seguito le problematiche che abbiamo trovato nello specifico.

Classe Player

- Notiamo che Player ha un deck e che model ha 4 deck. Non capiamo come mai visto che i player devono essere capaci di pescare da tutti e 4 i deck.
- Il metodo Draw: card non prende un input, ma è necessario che lo prenda per decidere quale delle 6 carte pescabili venga scelta.
- Non è chiaro cosa faccia il metodo Play in quanto non prende input e non viene specificato il tipo ritornato

Classe Controller

Move non è molto autoesplicativo e viene chiamato in diversi metodi senza chiarire esattamente cosa sia o che scopo abbia. Non è nemmeno segnato il tipo della variabile. Lo stesso vale per status controller.

La funzione initializePlayers prende Player come input, ma non crediamo sia corretto in quanto non ci sono ancora players inizializzati

Classe Model

Non viene specificato cosa sia StatusModel.

Da UML, Model dovrebbe interagire con i 4 deck e con la gameBoard senza avere un'istanza di nessuno di questi come segnato dalle frecce tratteggiate, ma nel grafico non è segnato in alcun modo come queste calssi interagiscano.

Crediamo che abbia a che fare con StatusModel ma nonostante questo non compare nessuna funzione che mostri come interagiscano queste classi.

Classe Deck

Essendo Shuffle e Draw dei metodi interni a Deck, non ha senso che ai metodi venga passato il deck stesso. Inoltre la freccia tratteggiata segnala che ci sia una dipendenza fra Deck e Card, mentre sarebbe corretta una freccia di associazione visto che deck chiaramente contiene delle carte.

Classe CardPosition

L'enum è molto interessante, ma non siamo riusciti a capire che senso abbia avere un valore per quando una carta è in mano ad un giocatore. Se si ha intenzione di tenere le carte pescate nel deck non ha senso flaggarle come "in mano" o "non in mano" se non si sa di chi siano. Se non si ha intenzione di tenerle nel deck una volta pescate non ha senso flaggarle come carte in mano.

Classe Card

Troviamo che la classe card sia troppo scarna per gestire tutte le interazioni e funzionalità delle carte, in particolare manca di identificatori e classificazione per le carte obiettivo e per capire in base a quale criterio calcolare i punti degli obiettivi. Sarebbe conveniente creare delle sottoclassi apposite per i vari tipi di carta.

Classe SideCard

Ottima classe per gestire le facce delle carte, la voce "both" nell'Enum Side sembra però essere superflua.

Classe Board

La struttura dati scelta per Board non è una delle più adeguate. Una matrice di Corners non è efficiente né dal punto di vista di spazio occupato né da quello di tempo di ricerca/posizionamento/ calcolo dei punti.

Si consiglia un grafo o una HashMap.

In ogni caso, se dovete decidere di mantenere una matrice consigliamo di collassare la funzioni SearchCardToLinkCornerColumn e SearchCardToLinkCornerLine in una sola. In particolare, invece che ritornare due int potrebbero ritornare un tipo Point.

Classe ScoreBoard

Non è chiaro a cosa serva il metodo Move. Sembra essere un metodo relativo alla view e non al game model.

Abbiamo interpretato il metodo Move (...) come un metodo che sposta un token di una certa distanza rispetto alla posizione in cui è. Essendo (come comunicato tramite mail) TokenPosition un array di int, crediamo che la funzione serva ad aumentare di "int distance" l'int relativo al token (così da muoverlo).

Visto che GameBoard è l'unica classe che può chiamare quel metodo, e visto che non ha uno storico dei punteggi ma solo i punteggi attuali, non c'è modo di calcolare "Distance". Inoltre "PositionToken" all'interno di Move non sembra essere necessario.

Un metodo setter sarebbe più adeguato.

Classe Token

Non è chiaro cosa faccia il metodo Associate(Player, Color), in quanto il metodo può solo chiamare funzioni pubbliche di Player e modificare dati all'interno di Token, e in nessuna di queste due classi è presente una variabile o un qualcosa che possa effettivamente associare il player al colore.

IsFirst all'interno sia di Token sia di Player è ridondante.

Confronto tra le architetture

Risulta difficile effettuare un confronto dettagliato tra le architetture in questione poiché la versione revisionata del design pattern MCV non è sufficientemente esaustiva da consentire un'analisi approfondita, soprattutto in mancanza di una spiegazione supplementare chiara. Questa circostanza potrebbe averci indotto in errore nel valutare il modello, motivando pertanto la richiesta di chiarimenti direttamente al gruppo di revisione tramite e-mail. Inoltre, le due architetture presentano differenze significative, rendendo quindi complesso il confronto diretto.

Tuttavia, è possibile notare una suddivisione delle funzionalità del gioco in macroaree coerenti, quali Model, Controller e View, aspetto che noi non abbiamo ancora implementato. In particolare, va apprezzata la gestione dell'enumerazione per i Corner, che si dimostra molto efficace e potrebbe essere d'ispirazione per la nostra architettura.

Come considerazione finale suggeriamo, anche per le future Peer Review, di adottare un approccio più chiaro e di fornire una documentazione più completa, o almeno una spiegazione più dettagliata. Sospettiamo infatti che il modello possa essere di buona qualità, ma purtroppo la sua comprensione è stata ostacolata dalla mancanza di chiarezza, che ha reso necessaria la formulazione di tali commenti.